ISLS Extension Phase 5.1 v1.0.0

C24: Recursive Decomposition and Hierarchical Composition Engine

Hypercubes of Hypercubes:
From Intent to System through
Atomic Crystallization and Upward Composition

Sebastian Klemm

16 March 2026

Abstract

This document specifies `isls-compose` (C24), a recursive decomposition and hierarchical composition engine that extends the ISLS Forge (C21–C23) from producing single artifacts to producing *systems of artifacts* — complete software components, services, or multi-module architectures.

The core mechanism: a top-level DecisionSpec is recursively decomposed into atomic sub-specs. Each atom is independently drilled through PMHD (C21), crystallized through the 8-gate cascade, and emitted as an ArtifactIR (C22). The atomic crystals are then composed upward through interface resolution, dependency validation, and hierarchical crystallization — producing a *System Crystal* that contains the complete artifact tree with full provenance at every level.

Each crystal at every level is a Hypercube Universe (C18). Interfaces between components are Bridges. The atom-molecule-system hierarchy uses Scale Ladders. The entire recursive process is deterministic, replay-verified, and evidence-bound.

The result is a generative engine that can produce arbitrarily complex structured outputs — not by rendering templates, but by recursive adversarial crystallization from intent down to atoms and composition back up to systems.

Status: Implementation Specification.

Parent: ISLS v1.0.0 + Extensions Phase 1–5.

Depends on: C18 (Scale), C21 (PMHD), C22 (ArtifactIR), C23 (Forge).

Contents

I Foundations 4

1 The Problem with Flat Synthesis 4

2 Axioms 4

3 Three Levels 4

II Decomposition 5

4 The Decomposition Tree 5

5 Decomposition Algorithm 5

6 Decomposition Strategies 6

III Per-Atom Forging 6

7 Atomic Forge Pipeline 6

8 Parallel Forging 7

IV Composition 7

9 Interface Resolution 7

10 Upward Crystallization 8

11 Re-Forging on Failure 9

V Synthesized Output 9

12 System Artifact Assembly 9

VI Scale Integration 10

13 Mapping to Hypercube Universes 10

VII Configuration and API 11

14 Configuration 11

15 Rust API 11

16 CLI Integration 13

17 Gateway Integration **13**

VIII Acceptance Tests **13**

IX Dependency Graph and Handoff **14**

18 Dependency Graph **14**

19 Test Count **15**

20 Completion Criteria **15**

Part I
Foundations

1 The Problem with Flat Synthesis

Phase 5 produces single artifacts: one DecisionSpec → one Monolith → one ArtifactIR → one output. This is sufficient for generating a function, a schema, or a configuration file. It is not sufficient for generating a *system*: a multi-component architecture with internal interfaces, dependency ordering, layered abstractions, and compositional correctness.

A system is not a large artifact. It is a *tree of artifacts* where each node is independently valid and the tree as a whole satisfies compositional constraints that no single node can express.

2 Axioms

Axiom 2.1 (Recursive Decomposability)**.** Any DecisionSpec that describes a system can be decomposed into a finite tree of sub-specs, where each leaf spec describes an atomic component that is synthesizable by a single Forge run.

Axiom 2.2 (Independent Atomic Validity)**.** Each atomic component must independently pass the 8-gate validation. Compositional validity does not compensate for atomic invalidity.

Axiom 2.3 (Compositional Closure)**.** The composition of individually valid components into a system must itself be validated through the 8-gate cascade. The system crystal is not merely the union of its parts — it is a new crystal that proves the composition is sound.

Axiom 2.4 (Interface-First Decomposition)**.** Decomposition is driven by interfaces, not by content. The decomposition algorithm identifies *what needs to connect to what* before deciding *what each component does*. Interfaces are the invariant; implementations are the variables.

Axiom 2.5 (Scale Correspondence)**.** The decomposition hierarchy maps directly to the Scale architecture (C18): atoms are micro-scale Hypercube Universes, modules are meso-scale clusters, and the system is the macro-scale domain. Bridges between atoms are component interfaces. Ladders between levels are the composition operators.

3 Three Levels

Definition 3.1 (Atom)**.** An atom is a leaf-level component: a single function, type, endpoint, schema definition, or configuration block. An atom has no internal sub-components. It is the smallest unit that can be independently drilled, crystallized, and validated.

Definition 3.2 (Molecule)**.** A molecule is a composition of atoms that together form a coherent module: a service, a library, a layer. A molecule has internal interfaces (between its atoms) and external interfaces (exposed to other molecules or to the system).

Definition 3.3 (System)**.** A system is a composition of molecules that together form a complete deliverable: an application, a service mesh, a platform component. The system crystal is the root of the composition tree.

Remark 3.1. The three levels are not fixed. For very complex systems, the hierarchy may have more than three levels (atoms → molecules → assemblies → systems → platforms). The decomposition engine supports arbitrary depth. Three levels is the minimum useful hierarchy.

Part II
Decomposition

4 The Decomposition Tree

Definition 4.1 (CompositionTree)**.** A CompositionTree $\mathcal{T}$ is a rooted tree where:

- The root is the top-level DecisionSpec $\mathcal{D}_0$.
- Each internal node is a DecisionSpec $\mathcal{D}_i$ with children $\{\mathcal{D}_{i,1}, \ldots, \mathcal{D}_{i,k_i}\}$.
- Each leaf is an atomic DecisionSpec that is directly forge-able by C23.
- Each edge carries an InterfaceContract specifying what the child provides to or requires from its parent and siblings.

Definition 4.2 (InterfaceContract)**.** An InterfaceContract $\mathcal{I}$ between two nodes specifies:

$$\mathcal{I} = \langle \texttt{provider}, \texttt{consumer}, \texttt{provides}, \texttt{requires}, \texttt{protocol}, \texttt{direction} \rangle$$

where:

- `provides`: list of named capabilities with type signatures (e.g., `"authenticate(token) -> User"`),
- `requires`: list of named dependencies with type signatures,
- `protocol`: interaction pattern (`sync_call`, `async_message`, `shared_type`, `event_stream`),
- `direction`: `Unidirectional` or `Bidirectional`.

5 Decomposition Algorithm

Algorithm 1 DECOMPOSE: Recursive spec decomposition

Require: Top-level DecisionSpec $\mathcal{D}_0$, decomposition config θ
Ensure: CompositionTree $\mathcal{T}$

```
1: T ← NewTree(D_0)
2: DecomposeNode(T, D_0, 0)
3: return T
```

Algorithm 2 DECOMPOSENODE: Recursive node decomposition

Require: Tree $\mathcal{T}$, spec $\mathcal{D}$, depth d

```
1: if IsAtomic(D, θ) ∨ d ≥ θ.max_depth then
2:     Mark D as leaf in T
3:     return
4: end if
5: {(D_1, I_1), ..., (D_k, I_k)} ← IdentifyComponents(D, θ)
6: for j = 1 to k do
7:     Add D_j as child of D in T with edge I_j
8:     DecomposeNode(T, D_j, d + 1)
9: end for
```

Definition 5.1 (Atomicity Test)**.** A DecisionSpec is atomic if and only if:

(i) it describes a single responsibility (one primary capability),
(ii) its estimated component count is $\leq \theta_{\text{atom_max}}$ (default: 5 components in the ArtifactIR),

(iii) it has no natural internal interface boundaries,
(iv) or the depth limit has been reached.

Definition 5.2 (Component Identification)**.** Component identification analyzes a non-atomic DecisionSpec and splits it along interface boundaries:

(i) **Capability clustering**: group related capabilities (e.g., all auth-related capabilities form one component).
(ii) **Dependency analysis**: identify which capabilities depend on which others; cut at minimal-dependency boundaries.
(iii) **Interface extraction**: for each cut, define the InterfaceContract (what crosses the boundary).
(iv) **Sub-spec generation**: create a new DecisionSpec for each cluster, inheriting relevant goals and constraints from the parent, plus new constraints from the interface contracts.

The identification is deterministic: same spec and config produce identical decomposition.

6 Decomposition Strategies

Definition 6.1 (Decomposition Strategies)**.** The decomposition engine supports multiple strategies for splitting a spec into sub-specs:

(i) **LayerDecomp**: split by architectural layers (e.g., data layer, business logic, API layer, presentation).
(ii) **FeatureDecomp**: split by features / capabilities (each feature becomes a component).
(iii) **DomainDecomp**: split by domain boundaries (Domain-Driven Design-style bounded contexts).
(iv) **PipelineDecomp**: split by processing stages (input → transform → validate → output).
(v) **HybridDecomp**: combine strategies per level (e.g., LayerDecomp at system level, FeatureDecomp at module level).
(vi) **CustomDecomp**: user-provided decomposition function.

All strategies are deterministic.

Part III
Per-Atom Forging

7 Atomic Forge Pipeline

Each leaf in the CompositionTree runs through the standard Phase 5 forge pipeline independently:

1. PMHD Drill on the atomic DecisionSpec → Monolith
2. ArtifactIR from Monolith
3. Matrix interpretation (domain-specific)
4. Synthesis → raw output
5. Evaluation against constraints
6. 8-gate validation → Atomic Crystal C_{atom}

Invariant 7.1 (Atomic Independence)**.** Each atomic forge run is independent: it does not read the output of sibling atoms. It knows about siblings only through the InterfaceContracts attached to its DecisionSpec. This ensures that each atom can be forged in parallel and that atomic validity is self-contained.

Requirement 7.1 (Interface Constraint Injection)**.** When an atom's DecisionSpec is created by decomposition, the InterfaceContracts from its parent and siblings are injected as additional hard constraints:

- `provides` entries become "must export" constraints.
- `requires` entries become "must import" constraints.
- Type signatures from contracts become compatibility constraints.

This ensures that independently forged atoms produce compatible interfaces.

8 Parallel Forging

Definition 8.1 (Forge Schedule)**.** The forge schedule determines the order of atomic forge runs. Since atoms are independent (by invariant 7.1), all atoms at the same depth level can be forged in parallel. The schedule is a sequence of *waves*:

$$W_d = \{C_{\text{atom}} : \text{depth}(C) = d_{\max}\}$$

Wave $d_{\max}$ (deepest leaves) is forged first, then $d_{\max} - 1$, up to the root. Within each wave, all atoms are independent and may be forged concurrently.

Requirement 8.1 (Sequential Fallback)**.** On single-core systems, atoms within a wave are forged sequentially in deterministic order (sorted by sub-spec content address). This preserves determinism regardless of parallelism.

Part IV
Composition

9 Interface Resolution

Definition 9.1 (Interface Resolution)**.** After all atoms in a molecule are forged, their interfaces are resolved:

$$\text{Resolve}(\{C_1, \ldots, C_k\}, \{\mathcal{I}_1, \ldots, \mathcal{I}_m\}) = (\Gamma, \Delta)$$

where Γ is the set of *satisfied* interface bindings and Δ is the set of *unsatisfied* requirements.

Definition 9.2 (Interface Binding)**.** A binding $(p, c, \mathcal{I})$ is satisfied when:

(i) Component p declares a `provides` entry matching $\mathcal{I}$.`provides`.
(ii) Component c declares a `requires` entry matching $\mathcal{I}$.`requires`.
(iii) The type signatures are compatible (structural subtyping: the provider's signature is at least as general as the consumer's expectation).
(iv) The protocol matches.

Definition 9.3 (Resolution Failure Modes)**.**

(i) **Unsatisfied requirement**: a consumer needs something no provider offers. The composition cannot proceed.
(ii) **Type mismatch**: provider and consumer have incompatible type signatures. The offending atom must be re-forged with corrected constraints.
(iii) **Circular dependency**: the dependency graph among atoms contains a cycle. The decomposition must be revised.
(iv) **Ambiguous binding**: multiple providers match a single requirement. Resolution uses deterministic tie-breaking (provider with highest quality score, then by content address).

Requirement 9.1 (Resolution is Deterministic)**.** Interface resolution **MUST** produce identical bindings under identical inputs. All tie-breaking rules are deterministic and specified in the Run Descriptor.

10 Upward Crystallization

Definition 10.1 (Molecule Crystal)**.** A molecule crystal C_{mol} is produced by composing its atomic crystals:

$$C_{\text{mol}} = \text{Compose}(\{C_{\text{atom},1}, \ldots, C_{\text{atom},k}\}, \Gamma, \mathcal{D}_{\text{mol}})$$

The molecule crystal contains:

- `sub_crystal_ids`: content addresses of all atomic crystals,
- `interface_bindings`: the resolved binding set Γ,
- `composition_proof`: evidence that all interfaces are satisfied and the dependency graph is acyclic,
- `constraint_program`: the union of atomic constraint programs plus compositional constraints,
- its own `topology_signature`: the dependency graph topology of its atoms (using Betti numbers, spectral gap, etc. from C16).

Definition 10.2 (Composition Validation)**.** The molecule crystal passes through a dedicated composition validation before the standard 8-gate cascade:

(CV1) **All atoms valid**: every sub-crystal passes 8/8 checks.
(CV2) **No unsatisfied interfaces**: $\Delta = \emptyset$.
(CV3) **No type mismatches**: all bindings type-check.
(CV4) **Acyclic dependencies**: topological sort succeeds.
(CV5) **Coverage**: every capability from the parent DecisionSpec is provided by at least one atom.
(CV6) **No orphans**: every atom provides at least one capability that is consumed.

After CV1–CV6 pass, the molecule crystal enters the standard 8-gate cascade.

Definition 10.3 (System Crystal)**.** The system crystal C_{sys} is produced by composing molecule crystals. Its structure is identical to a molecule crystal but at one level higher:

- `sub_crystal_ids`: content addresses of molecule crystals,
- each molecule crystal itself references its atomic crystals,
- the system crystal's `topology_signature` describes the inter-module dependency graph.

The system crystal passes CV1–CV6 on its molecule children, then the 8-gate cascade.

Theorem 10.1 (Hierarchical Validity)**.** *If the system crystal passes CV1–CV6 and the 8-gate cascade, then every crystal at every level of the hierarchy is independently valid: every atom passes 8/8, every molecule passes CV1–CV6 + 8/8, and the system passes CV1–CV6 + 8/8. The validity is not inherited downward — it is independently established at each level and verified compositionally at each parent level.*

Proof. By construction: CV1 at the system level requires that all molecule sub-crystals pass 8/8. CV1 at the molecule level requires that all atomic sub-crystals pass 8/8. The 8-gate cascade at each level operates on the crystal's own fields independently. Therefore validity at level n implies validity at all levels $< n$, and each level's validity is independently checkable. □

11 Re-Forging on Failure

Definition 11.1 (Composition Repair)**.** When interface resolution fails (type mismatch or unsatisfied requirement), the composition engine can attempt repair:

(i) **Constraint refinement**: add the failing interface requirement as an explicit constraint to the offending atom's DecisionSpec and re-forge.

(ii) **Adapter synthesis**: forge a new "glue" atom whose sole purpose is to bridge the type mismatch between two incompatible atoms.

(iii) **Decomposition revision**: re-run decomposition with different boundaries if multiple atoms fail to compose.

Requirement 11.1 (Repair Budget)**.** The repair loop **MUST** be bounded by a configurable budget (max re-forge attempts per atom, max adapter atoms, max decomposition revisions). When the budget is exhausted, the composition fails with a detailed diagnostic listing all unresolved interfaces.

Part V
Synthesized Output

12 System Artifact Assembly

Definition 12.1 (SystemArtifact)**.** The final output of a composition run is a SystemArtifact:

```
pub struct SystemArtifact {
    pub system_crystal: SemanticCrystal,
    pub tree: CompositionTree,
    pub atoms: Vec<AtomArtifact>,
    pub molecules: Vec<MoleculeArtifact>,
    pub bindings: Vec<InterfaceBinding>,
    pub manifest: ExecutionManifest,
    pub total_components: usize,
    pub total_interfaces: usize,
    pub depth: usize,
}

pub struct AtomArtifact {
    pub crystal: SemanticCrystal,
    pub ir: ArtifactIR,
    pub synthesis: SynthesisOutput,
    pub file_path: String,
}

pub struct MoleculeArtifact {
    pub crystal: SemanticCrystal,
    pub atom_ids: Vec<Hash256>,
    pub bindings: Vec<InterfaceBinding>,
    pub composition_proof: CompositionProof,
}
```

Definition 12.2 (File Layout)**.** The emitter produces a file tree reflecting the composition hierarchy:

```
output/
  system.crystal.json          # System crystal
  manifest.json                # Execution manifest
  composition_proof.json       # CV1-CV6 evidence
  modules/
    auth/
      auth.crystal.json        # Molecule crystal
      atoms/
        auth_handler.rs        # Atom artifact
        auth_handler.crystal.json
        token_validator.rs
        token_validator.crystal.json
    database/
      database.crystal.json
      atoms/
        schema.sql
        schema.crystal.json
        connection.rs
        connection.crystal.json
    api/
      api.crystal.json
      atoms/
        routes.rs
        routes.crystal.json
        middleware.rs
        middleware.crystal.json
  interfaces/
    bindings.json              # All resolved interface bindings
    dependency_graph.json      # Topological order
```

Every file has a companion crystal. Every crystal is independently verifiable. The system crystal references all others.

Part VI
Scale Integration

13 Mapping to Hypercube Universes

Definition 13.1 (Composition-Scale Mapping)**.** The composition hierarchy maps to the C18 Scale architecture:

- Each atom is a micro-scale Hypercube Universe. Its vertices are the ArtifactIR components. Its edges are component dependencies. Its aggregate state is the atom's 5D quality signature.
- Each molecule is a meso-scale Hypercube Universe. Its vertices are the atom-universes. Its edges are interface bindings (Bridges). Its aggregate state is the molecule's composite quality.
- The system is the macro-scale Hypercube Universe. Its vertices are molecule-universes. Its edges are inter-module interfaces.

Definition 13.2 (Composition Bridges)**.** An interface binding between two atoms becomes a Bridge (C18) between their Hypercube Universes:

- Weight = type compatibility score,
- Delay = 0 (synchronous composition),
- Phase offset = 0,
- Activation predicate = always active (bindings are resolved, not conditional).

Definition 13.3 (Composition Ladders)**.** The Lift and Projection operators from C18 apply:

- $\text{Lift}_{\text{atom}\to\text{mol}}$: aggregate atom quality metrics into molecule quality.
- $\text{Proj}_{\text{mol}\to\text{atom}}$: propagate molecule-level constraints down to atoms (for re-forging).
- $\text{Lift}_{\text{mol}\to\text{sys}}$: aggregate molecule quality into system quality.

Part VII
Configuration and API

14 Configuration

Composition Configuration

```
compose:
  max_depth: 4                # max decomposition levels
  atom_max_components: 5      # atomicity threshold
  decomposition_strategy: "hybrid"
  parallel_forge: false       # true if multi-core
  repair:
    max_reforge_per_atom: 3
    max_adapter_atoms: 5
    max_decomp_revisions: 2
  quality_thresholds:
    min_atom_quality: 0.5     # minimum per-axis quality for atoms
    min_molecule_cv_score: 0.7
    min_system_coverage: 0.9
```

15 Rust API

```
// isls-compose/src/lib.rs

pub struct CompositionTree {
    pub root: TreeNode,
    pub depth: usize,
    pub atom_count: usize,
    pub molecule_count: usize,
}

pub struct TreeNode {
    pub spec: DecisionSpec,
    pub level: CompLevel,
    pub children: Vec<TreeNode>,
    pub interfaces: Vec<InterfaceContract>,
    pub crystal: Option<SemanticCrystal>,
}
```

```
pub enum CompLevel { System, Molecule, Atom }

pub struct InterfaceContract {
    pub provider: String,
    pub consumer: String,
    pub provides: Vec<Capability>,
    pub requires: Vec<Capability>,
    pub protocol: Protocol,
    pub direction: Direction,
}

pub struct Capability {
    pub name: String,
    pub signature: String,     // type signature
    pub description: String,
}

pub enum Protocol { SyncCall, AsyncMessage, SharedType, EventStream }
pub enum Direction { Unidirectional, Bidirectional }

pub struct InterfaceBinding {
    pub provider_crystal_id: Hash256,
    pub consumer_crystal_id: Hash256,
    pub contract: InterfaceContract,
    pub compatibility_score: f64,
}

pub struct CompositionProof {
    pub cv_results: [bool; 6],    // CV1-CV6
    pub all_atoms_valid: bool,
    pub unsatisfied: Vec<String>, // empty if success
    pub dependency_order: Vec<Hash256>,
    pub coverage: f64,
}

pub struct CompositionEngine {
    forge: ForgeEngine,
    config: ComposeConfig,
}

impl CompositionEngine {
    pub fn new(forge: ForgeEngine, config: ComposeConfig) -> Self;

    /// Full pipeline: decompose -> forge atoms -> resolve -> compose ->
        validate
    pub fn compose(&mut self, spec: DecisionSpec) -> Result<
       SystemArtifact>;

    /// Decompose only (for inspection before forging)
    pub fn decompose(&self, spec: &DecisionSpec) -> Result<
       CompositionTree>;

    /// Forge all atoms in a tree (after decomposition)
    pub fn forge_atoms(&mut self, tree: &mut CompositionTree) -> Result<
       Vec<AtomArtifact>>;
```

```
    /// Resolve interfaces between forged atoms
    pub fn resolve_interfaces(
        &self, atoms: &[AtomArtifact], contracts: &[InterfaceContract]
    ) -> Result<(Vec<InterfaceBinding>, Vec<String>)>;

    /// Compose atoms into molecules, molecules into system
    pub fn compose_upward(
        &mut self, tree: &mut CompositionTree, atoms: &[AtomArtifact]
    ) -> Result<SystemArtifact>;

    /// Attempt repair on failed composition
    pub fn repair(
        &mut self, tree: &mut CompositionTree, failures: &[String]
    ) -> Result<SystemArtifact>;
}
```

16 CLI Integration

```
# Compose a full system from a DecisionSpec
isls compose --spec system-intent.json --domain rust --output system/

# Decompose only (inspect the tree before forging)
isls compose --spec system-intent.json --decompose-only

# Compose with specific strategy
isls compose --spec intent.json --strategy layer --max-depth 3

# Compose from existing crystals (re-compose with new interfaces)
isls compose --crystals crystal1.json,crystal2.json --output composed/
```

17 Gateway Integration

New Gateway Endpoints

```
POST /compose          Body: {spec, domain, config}
                       -> SystemArtifact with full crystal tree

POST /compose/decompose  Body: {spec, strategy}
                         -> CompositionTree (preview, no forging)

GET  /compose/status     -> current composition progress
                            (atoms forged / total, molecules composed)
```

Part VIII
Acceptance Tests

ID	Name	Procedure
AT-CO1	Decomposition determinism	Decompose same spec twice; verify identical trees.
AT-CO2	Atom forging	Decompose into 3 atoms; forge each; verify 3 crystals all pass 8/8.
AT-CO3	Interface resolution	Forge 3 atoms with compatible interfaces; resolve; verify all bindings satisfied.
AT-CO4	Type mismatch detection	Forge 2 atoms with incompatible types; verify resolution fails with diagnostic.
AT-CO5	Upward crystallization	Compose 3 atoms into molecule; verify molecule crystal passes CV1–CV6 + 8/8.
AT-CO6	System crystal	Compose 2 molecules into system; verify system crystal references both molecules.
AT-CO7	Hierarchical validity	Validate system crystal; verify every sub-crystal at every level independently valid.
AT-CO8	Repair on failure	Create intentional type mismatch; verify repair re-forges the atom with corrected constraint.
AT-CO9	Depth limit	Set max_depth=1; verify decomposition produces only atoms (no molecules).
AT-CO10	File layout	Compose a 2-molecule system; verify file tree matches expected layout.
AT-CO11	Composition determinism	Compose same spec twice; verify identical system crystal ID.
AT-CO12	Scale mapping	Compose system; verify C18 Hypercube Universes created for atoms, molecules, and system.

Part IX
Dependency Graph and Handoff

18 Dependency Graph

```
isls-compose (C24)
    depends on:
      isls-pmhd (C21)          -- drill engine
      isls-artifact-ir (C22)   -- IR construction
      isls-forge (C23)         -- synthesis + evaluation
      isls-scale (C18)         -- Hypercube/Bridge/Ladder mapping
      isls-consensus (C5)      -- 8-gate validation
      isls-archive (C7)        -- crystal storage
      isls-manifest (C13)      -- execution manifest
```

```
      isls-store (C17)          -- persistence
      isls-types (C1)           -- foundation types
```

19 Test Count

Phase	New Tests	Cumulative
Phase 1–5	231	231
Phase 5.1 (C24)	12	$\geq$ 243

20 Completion Criteria

Phase 5.1 is complete when:

1. Decomposition produces deterministic trees from specs.
2. Each atom is independently forged and validated (8/8).
3. Interface resolution detects matches and mismatches.
4. Molecule crystals pass CV1–CV6 + 8/8.
5. System crystals reference all sub-crystals.
6. Repair loop handles type mismatches within budget.
7. File layout reflects composition hierarchy.
8. Scale mapping creates Hypercube Universes at all levels.
9. Total tests $\geq$ 243, zero warnings.
10. **24 Crates. One binary. Systems from intent.**

Generated for ISLS Extension Phase 5.1 v1.0.0.
Atoms crystallize. Molecules compose. Systems emerge.
Every level proven. Every interface bound. Every crystal verified.